SCHOOL OF ENGINEERING

23AM2403
Database Management System
Unit 2

SCHOOL OF ENGINEERING

Unit 2 Relational Model and SQL

UNIT – II	07 Hours

RELATIONAL MODEL :

Relational Model Concepts, Relational Model Constraints and Relational Database Schemas, Update operations and Dealing with Constraint Violations.

(Text Book-1: Chapter 3: 3.1 to 3.3).

SQL –THE RELATIONAL DATABASE STANDARD:

SQL Data Definition and Data types, Specifying constraints in SQL, Basic Queries in SQL-Data Definition Language in SQL, Data Manipulation Language in SQL;

(Text Book-1: Chapter 4: 4.1 to 4.4).

Relational Model Concepts

- The Relational Model of Data is based on the concept of a Relation
- The strength of the relational approach to data management comes from the formal foundation provided by the theory of relations
- We review the essentials of the formal relational model in this chapter
- In practice, there is a standard model based on SQL – this is described in the second part of Unit 2
- Note: There are several important differences between the formal model and the practical model, as we shall see .

Relational Model Concepts

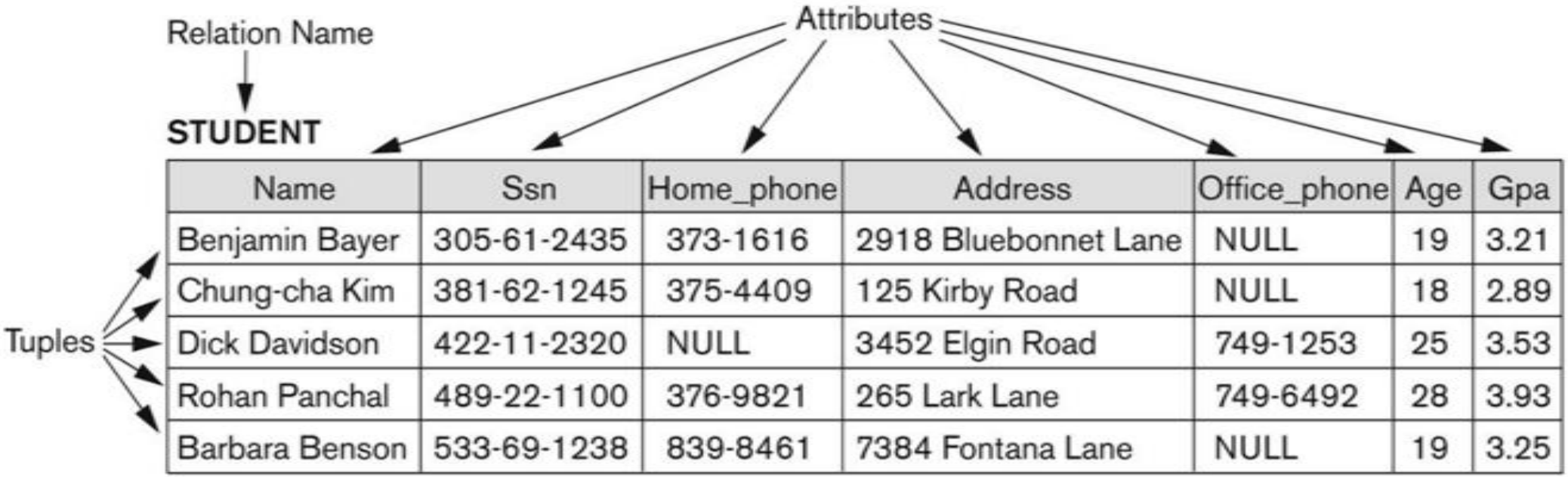
- A Relation is a mathematical concept based on the ideas of sets.
- The model was first proposed by Dr. E.F. Codd of IBM Research in 1970 in the following paper: "A Relational Model for Large Shared Data Banks," Communications of the ACM, June 1970 .
- The above paper caused a major revolution in the field of database management and earned Dr. Codd the coveted ACM Turing Award.

Informal Definitions

- Informally, a relation looks like a table of values.
- A relation typically contains a set of rows.
- The data elements in each row represent certain facts that correspond to a real-world entity or relationship
  - In the formal model, rows are called tuples
- Each column has a column header that gives an indication of the meaning of the data items in that column .
  - In the formal model, the column header is called an attribute name (or just attribute)

Example of a Relation

Attributes and Tuples of a relation Student



Name	Ssn	Home_phone	Address	Office_phone	Age	Gpa
Benjamin Bayer	305-61-2435	373-1616	2918 Bluebonnet Lane	NULL	19	3.21
Chung-cha Kim	381-62-1245	375-4409	125 Kirby Road	NULL	18	2.89
Dick Davidson	422-11-2320	NULL	3452 Elgin Road	749-1253	25	3.53
Rohan Panchal	489-22-1100	376-9821	265 Lark Lane	749-6492	28	3.93
Barbara Benson	533-69-1238	839-8461	7384 Fontana Lane	NULL	19	3.25

Formal Definitions - Domain

- A domain has a logical definition:
  - Example: “Mobile numbers” are the set of 10 digit phone numbers .
  - “Aadhar Card Numbers” are set of 12 digit numbers valid in India
- A domain also has a data-type or a format defined for it.
  - The USA_phone_numbers may have a format: (ddd)ddd-dddd where each d is a decimal digit.
  - Dates have various formats such as year, month, date formatted as yyyy-mm-dd, or as dd- mm-yyyy etc.

Some more Egs:

- Names: The set of names of persons.
- Grade_point_averages: Possible values of computed grade point averages; each must be a real (floating point) number between 0 and 4.
- Employee_ages: Possible ages of employees of a company; each must be a value between 15 and 80 years old.
- Academic_department_names: The set of academic department names, such as Computer Science, Economics, and Physics, in a university.
- Academic_department_codes: The set of academic department codes, such as CS, ECON, and PHYS, in a university.

Formal Definitions - Attribute

- The attribute name designates the **role** played by a domain in a relation.
- Used to **interpret** the meaning of the **data elements** corresponding to that attribute
  - Example: The domain Date may be used to define two attributes named “Invoice-date” and “Payment-date” with different meanings

Formal Definitions - Tuples

- A tuple is an ordered set of values enclosed in angled brackets '< … >'
- Each value is derived from an appropriate domain.
- A row in the CUSTOMER relation is a 4-tuple and would consist of four values, for example:

  <632895, "John Smith", "101 Main St. Atlanta, GA 30332",

  "(404) 894-2000">
- This is called a 4-tuple as it has 4 values
- A tuple is a row in the CUSTOMER relation.
- A relation is a set of such tuples (rows)

Formal Definition-Relation

- Formally,
  - Given $R(A_1, A_2, ..., A_n)$
  - $r(R) \subseteq$ **dom (A_1) X dom (A_2) X ....X dom(A_n)**
- $R(A_1, A_2, \ldots, A_n)$ is the **schema** of the relation
- R is the **name** of the relation
- $A_1, A_2, ..., A_n$ are the **attributes** of the relation
- r(R): a **specific state** (or "value" or “population”) of relation R – this is a set of tuples (rows)
- $r(R) = \{t_1, t_2, ..., t_n\}$ where each t_i is an n-tuple
- $t_i = < v_1, v_2, ..., v_n >$where each v_i element of domain(Aj)

Informal Terms	Formal Terms
Table Values	Relation
Column Header	Attribute
All possible Column Values	Domain
Row	Tuple
Table Definition	Schema of a Relation

Practice

a) Mention the no: of tuples in this relation.

b)Mention the no: of domains/attributes in this relation

emp_id	emp_name	emp_address
001	Claira Anderson	113, Zaraiah Road, TX 77001
003	Marc Doe	34343, Palm Jumeriah Road, Dubai 990039
005	Bruce Quilt	23, Santa Cruiz Road, NY 44303

Characteristics of Relation

- Ordering of tuples in a relation r(R):
  - The tuples are not considered to be ordered, even though they appear to be in the tabular form.
  - In other words, a relation is **not sensitive** to the ordering of tuples.

Eg: tuples in the STUDENT relation could be ordered by values of Name, Ssn, Age, or some other attribute.

STUDENT

Name	Ssn	Home_phone	Address	Office_phone	Age	Gpa
Benjamin Bayer	305-61-2435	(817)373-1616	2918 Bluebonnet Lane	NULL	19	3.21
Chung-cha Kim	381-62-1245	(817)375-4409	125 Kirby Road	NULL	18	2.89
Dick Davidson	422-11-2320	NULL	3452 Elgin Road	(817)749-1253	25	3.53
Rohan Panchal	489-22-1100	(817)376-9821	265 Lark Lane	(817)749-6492	28	3.93
Barbara Benson	533-69-1238	(817)839-8461	7384 Fontana Lane	NULL	19	3.25

STUDENT

Name	Ssn	Home_phone	Address	Office_phone	Age	Gpa
Dick Davidson	422-11-2320	NULL	3452 Elgin Road	(817)749-1253	25	3.53
Barbara Benson	533-69-1238	(817)839-8461	7384 Fontana Lane	NULL	19	3.25
Rohan Panchal	489-22-1100	(817)376-9821	265 Lark Lane	(817)749-6492	28	3.93
Chung-cha Kim	381-62-1245	(817)375-4409	125 Kirby Road	NULL	18	2.89
Benjamin Bayer	305-61-2435	(817)373-1616	2918 Bluebonnet Lane	NULL	19	3.21

Both relations are same ,but with different ordering of tuples

- t1 = < (Name, Dick Davidson),(Ssn, 422-11-2320),(Home_phone, NULL),(Address, 3452 Elgin Road), (Office_phone, (817)749-1253),(Age, 25),(Gpa, 3.53)>

- t2 = < (Address, 3452 Elgin Road),(Name, Dick Davidson),(Ssn, 422-11-2320),(Age, 25), (Office_phone, (817)749-1253),(Gpa, 3.53),(Home_phone, NULL)>

- At an abstract level, the order inside the tuples doesn’t matter.
- When the attribute name and value are included together in a tuple, it is known as self-describing data, because the description of each value (attribute name) is included in the tuple.

Characteristics of Relation

- **Values in the Tuples:**
- Each value in a tuple is an atomic value; that is, it is not divisible into components within the framework of the basic relational model.
- Each value in a tuple must be from the domain of the attribute for that column
- If tuple t = <v1, v2, …, vn> is a tuple (row) in the relation state r of R(A1, A2, …, An) , Then each vi must be a value from dom(Ai)
- We refer to component values of a tuple t by t[Ai] or t.Ai
- This is the value vi of attribute Ai for tuple t

Characteristics of Relation

- **Null values:** It is used to represent the values of attributes that may be unknown or may not apply to a tuple.

Several Meanings of NULL values
1)Value unknown
2) Value exists but is not available
3) Attribute does not apply
4) value undefined

NB: During database design, it is best to avoid NULL values as much as possible

Relational Model - Keys

- Primary Key
- Candidate Key
- Super Key
- Foreign Key
- Alternate Key

Relational Model - Keys

1. Primary Key (PK)

- A **Primary Key** is a unique identifier for a record in a table.
- It **must be unique** and **cannot have NULL values.**
- A table can have **only one primary key.**
- Example:

```sql
CREATE TABLE Students (
    StudentID INT PRIMARY KEY,
    Name VARCHAR(50),
    Age INT
);
```

- Here, `StudentID` is the **Primary Key.**

Relational Model - Keys

2. Candidate Key

- A **Candidate Key** is a set of attributes that can uniquely identify a record.
- A table can have **multiple candidate keys**, but only **one can be chosen as the Primary Key**.
- Candidate keys **cannot have NULL values**.
- Example:

sql Copy Edit

```
Table: Employees
+------------+------------+--------------+
| EmpID      | Email      | PhoneNumber  |
+------------+------------+--------------+
```

  - Possible **Candidate Keys**: `{EmpID}, {Email}, {PhoneNumber}`
  - Any of them could be chosen as the **Primary Key**.

Relational Model - Keys

3. Alternative Key

- **Alternative Keys** are Candidate Keys **that are not chosen as the Primary Key.**
- They still have unique constraints but are not used as the primary identifier.
- Example:
  - From the above example, if **EmpID** is the **Primary Key**, then **Email** and **PhoneNumber** are **Alternative Keys**.

Relational Model - Keys

4. Foreign Key (FK)

- A **Foreign Key** is an attribute in a table that **refers to the Primary Key of another table.**
- It **establishes relationships** between tables.
- Foreign Keys **can have duplicate values.**
- Example:

sql Copy Edit

```sql
CREATE TABLE Orders (
    OrderID INT PRIMARY KEY,
    StudentID INT,
    FOREIGN KEY (StudentID) REFERENCES Students(StudentID)
);
```

- Here, `StudentID` in the `Orders` table is a **Foreign Key** referring to `StudentID` in the `Students` table.

Relational Model - Keys

5. Super Key

- A **Super Key** is a set of one or more attributes that can uniquely identify a record.
- It **includes** all Candidate Keys and the Primary Key.
- A **Super Key can have extra attributes that are not necessary for uniqueness.**
- Example:
  - `{EmpID}`, `{EmpID, Email}`, `{EmpID, PhoneNumber, Name}` are all **Super Keys** because they can uniquely identify a record.

Relational Model - Keys

Key Type	Definition	Unique?	NULL Allowed?	Example
Primary Key	Uniquely identifies a record in a table	✅ Yes	❌ No	`StudentID` in `Students` table
Candidate Key	A set of attributes that can uniquely identify a record	✅ Yes	❌ No	`{EmpID}`, `{Email}`, `{PhoneNumber}` in `Employees`
Alternative Key	A Candidate Key that was not chosen as the Primary Key	✅ Yes	❌ No	`Email` if `EmpID` is Primary Key
Foreign Key	An attribute that references the Primary Key of another table	❌ No	✅ Yes (unless constrained)	`StudentID` in `Orders` referencing `Students`
Super Key	A set of attributes that uniquely identify a record, including Candidate and Primary Keys	✅ Yes	❌ No	`{EmpID, Name}`, `{EmpID, Email}`

Relational Model Constraints

SCHOOL OF ENGINEERING

Relational Model Constraints in DBMS

In a **Relational Database Management System (RDBMS)**, constraints are rules that ensure data integrity, accuracy, and consistency. The **Relational Model** defines several key constraints to maintain the correctness of data.

Types of Relational Model Constraints

1. **Domain Constraints**
2. **Key Constraints**
3. **Entity Integrity Constraints**
4. **Referential Integrity Constraints**
5. **Semantic Constraints (User-defined constraints)**

Relational Model Constraints

SCHOOL OF ENGINEERING

1. Domain Constraints

- **Definition:** Domain constraints specify that the values of attributes must belong to a defined data type or domain.
- Each column (attribute) must have a valid data type such as `INT`, `VARCHAR`, `DATE`, etc.
- Example:

sql Copy Edit

```sql
CREATE TABLE Students (
    Student_ID INT NOT NULL,
    Name VARCHAR(50),
    Age INT CHECK (Age >= 18)
);
```

- In this example, `Age` must be 18 or above.

Relational Model Constraints

SCHOOL OF ENGINEERING

2. Key Constraints

- **Definition:** A key constraint ensures that each tuple (row) in a table is uniquely identifiable.
- **Primary Key** and **Unique Key** constraints are used to enforce this rule.

✅ **Primary Key (PK)**

- Ensures unique and non-null values.
- Each table can have only **one primary key**.

✅ **Unique Key**

- Ensures unique values but allows **null values**.
- A table can have **multiple unique keys**.

Relational Model Constraints

SCHOOL OF ENGINEERING

Example:

sql

Copy Edit

```sql
CREATE TABLE Employees (
    Emp_ID INT PRIMARY KEY,
    Email VARCHAR(100) UNIQUE,
    Phone_Number VARCHAR(15) UNIQUE
);
```

Relational Model Constraints

SCHOOL OF ENGINEERING

3. Entity Integrity Constraint

- **Definition:** Ensures that the **Primary Key** cannot contain **NULL** values.
- Entity integrity ensures that every row in a table can be uniquely identified.

Example:

sql Copy Edit

```sql
CREATE TABLE Orders (
    Order_ID INT PRIMARY KEY,
    Order_Date DATE NOT NULL
);
```

- Here, `Order_ID` must have a valid non-null value.

Relational Model Constraints

SCHOOL OF ENGINEERING

4. Referential Integrity Constraint

- **Definition:** Ensures that a **foreign key** value in one table matches a **primary key** value in another table.
- This maintains consistency between related tables.

✅ **Foreign Key (FK)**

- Establishes a relationship between two tables.
- Ensures data dependency and prevents invalid data entry.

Relational Model Constraints

SCHOOL OF ENGINEERING

Example:

sql

Copy Edit

```
CREATE TABLE Departments (
    Dept_ID INT PRIMARY KEY,
    Dept_Name VARCHAR(100)
);

CREATE TABLE Employees (
    Emp_ID INT PRIMARY KEY,
    Emp_Name VARCHAR(50),
    Dept_ID INT,
    FOREIGN KEY (Dept_ID) REFERENCES Departments(Dept_ID)
);
```

- The `Dept_ID` in the `Employees` table must match a valid `Dept_ID` in the `Departments` table.

Relational Model Constraints

SCHOOL OF ENGINEERING

5. Semantic Constraints (User-defined Constraints)

- **Definition:** These are business rules or conditions defined to enforce application-specific constraints.
- Example conditions include:
  - Salary must be greater than a certain limit.
  - Age must be between 18 and 60 for employees.

Example:

sql Copy Edit

```sql
CREATE TABLE Employees (
    Emp_ID INT PRIMARY KEY,
    Name VARCHAR(50),
    Age INT CHECK (Age >= 18 AND Age <= 60),
    Salary DECIMAL(10, 2) CHECK (Salary >= 20000)
);
```

Relational Model Constraints

SCHOOL OF ENGINEERING

Constraint Type	Description	Example
Domain Constraint	Ensures attribute values are within a specified range or type	`CHECK (Age >= 18)`
Key Constraint	Ensures each row is uniquely identified	`PRIMARY KEY (Emp_ID)`
Entity Integrity	Ensures primary keys are non-null	`NOT NULL`
Referential Integrity	Ensures foreign key matches primary key in another table	`FOREIGN KEY (Dept_ID) REFERENCES Departments(Dept_ID)`
Semantic Constraint	Custom rules for specific conditions	`CHECK (Salary >= 20000)`

Relational Database Schema

Relational Model Schema in DBMS

A **Relational Model Schema** in a **Database Management System (DBMS)** is a formal structure that defines the logical design of a relational database. It describes the organization of data in tables, their attributes, and the relationships between them.

Key Components of a Relational Model Schema

A relational schema is composed of:

1. **Relation (Table)**
2. **Attributes (Columns)**
3. **Tuples (Rows)**
4. **Domains (Data Types)**
5. **Keys (Primary, Foreign, etc.)**
6. **Integrity Constraints**
7. **Relations and Relationships**

Relational Database Schema

Key Characteristics of Relational Model Schema

✅ **Data Independence:** Changes to the database schema do not affect the application.

✅ **Logical Structure:** Focuses on representing data in tables with rows and columns.

✅ **Declarative Language Support:** Uses SQL to define schemas, constraints, and retrieve data efficiently.

✅ **Normalization:** Organizes data to minimize redundancy and improve data integrity.

Relational Database Schema

1. Relation (Table)

- A **relation** is a table that holds data in **rows** (tuples) and **columns** (attributes).
- Each relation is uniquely identified by a **relation name**.

Example:

```
STUDENT (Student_ID, Name, Age, Course, Marks)
```

Relational Database Schema

2. Attributes (Columns)

- An **attribute** is a column that represents a specific property of an entity.
- Each attribute has a **name** and a **domain** (data type).

Example:

In the `STUDENT` table:

- `Student_ID` : Integer
- `Name` : VARCHAR(50)
- `Age` : Integer
- `Course` : VARCHAR(30)
- `Marks` : Decimal(5,2)

Relational Database Schema

3. Tuples (Rows)

- A **tuple** is a single row or record in a relation.
- Each tuple represents one instance of the entity.

Example:

Student_ID	Name	Age	Course	Marks
101	John	20	Physics	85.5
102	Alice	22	Maths	90.0

Relational Database Schema

SCHOOL OF ENGINEERING

4. Domains (Data Types)

- A **domain** is a set of valid values for an attribute.
- Each attribute is associated with a particular domain to ensure data integrity.

Example:

- `Age` → Domain: Positive Integers (≥ 18)
- `Marks` → Domain: Decimal values between 0-100

---

5. Keys

- **Primary Key (PK):** Uniquely identifies each tuple in the table.
- **Foreign Key (FK):** Establishes relationships between tables.
- **Candidate Key:** A set of attributes that uniquely identify a row.
- **Super Key:** A superset of the candidate key that can uniquely identify each row.

Relational Database Schema

SCHOOL OF ENGINEERING

6. Integrity Constraints

- Integrity constraints are rules that maintain the correctness of data in the database.

✅ **Entity Integrity** → Primary key cannot be NULL.

✅ **Referential Integrity** → Ensures foreign key values match primary key values.

✅ **Domain Constraint** → Ensures attribute values are within valid ranges.

✅ **CHECK Constraint** → Ensures attribute conditions are satisfied.

Example:

sql Copy Edit

```sql
CREATE TABLE ENROLLMENT (
    Enroll_ID INT PRIMARY KEY,
    Student_ID INT,
    Course_ID INT,
    FOREIGN KEY (Student_ID) REFERENCES STUDENT(Student_ID),
    CHECK (Course_ID > 100)
);
```

Relational Database Schema

7. Relations and Relationships

- **One-to-One:** Each record in Table A maps to exactly one record in Table B.
- **One-to-Many:** Each record in Table A maps to multiple records in Table B.
- **Many-to-Many:** Records in Table A map to multiple records in Table B and vice versa.

Example of One-to-Many Relationship:

`STUDENT` → `ENROLLMENT`

One student can enroll in multiple courses.

Relational Database Schema

Example of a Relational Schema Design

Relational Model for a Library Management System

scss Copy Edit

```
BOOK (Book_ID, Title, Author, Publisher, Price)
STUDENT (Student_ID, Name, Age, Course)
BORROW (Borrow_ID, Student_ID, Book_ID, Borrow_Date, Return_Date)
```

Key Definitions:

- `Book_ID` → Primary Key in `BOOK`
- `Student_ID` → Primary Key in `STUDENT`
- `Borrow_ID` → Primary Key in `BORROW`
- `Student_ID` and `Book_ID` in `BORROW` are Foreign Keys referencing `STUDENT` and `BOOK`.

Relational Database Schema

SCHOOL OF ENGINEERING

SQL Implementation:

sql

Copy Edit

```sql
CREATE TABLE BOOK (
    Book_ID INT PRIMARY KEY,
    Title VARCHAR(100) NOT NULL,
    Author VARCHAR(50),
    Publisher VARCHAR(50),
    Price DECIMAL(8, 2)
);

CREATE TABLE STUDENT (
    Student_ID INT PRIMARY KEY,
    Name VARCHAR(50),
    Age INT CHECK (Age >= 18),
    Course VARCHAR(30)
);

CREATE TABLE BORROW (
    Borrow_ID INT PRIMARY KEY,
    Student_ID INT,
    Book_ID INT,
    Borrow_Date DATE,
    Return_Date DATE,
    FOREIGN KEY (Student_ID) REFERENCES STUDENT(Student_ID),
    FOREIGN KEY (Book_ID) REFERENCES BOOK(Book_ID)
);
```

Update Operations on Relations

SCHOOL OF ENGINEERING

Update Operations and Dealing with Constraint Violations in the Relational Model

In a **Relational Database Management System (RDBMS)**, data manipulation is achieved using **update operations**. These operations include **INSERT**, **DELETE**, and **UPDATE** statements. Each of these operations must adhere to various constraints to maintain data integrity and consistency.

---

1. Types of Update Operations in Relational Model

The primary update operations are:

1. **INSERT** → Adds new tuples (records) into a relation.
2. **DELETE** → Removes existing tuples from a relation.
3. **UPDATE** → Modifies the values of one or more attributes in existing tuples.

Update Operations on Relations

SCHOOL OF ENGINEERING

2. Handling Constraint Violations During Update Operations

Each update operation may violate constraints such as:

1. Domain Constraint Violation
2. Key Constraint Violation
3. Entity Integrity Constraint Violation
4. Referential Integrity Constraint Violation

Update Operations on Relations

SCHOOL OF ENGINEERING

3. Detailed Explanation with Examples

1. INSERT Operation

The `INSERT` command is used to add new records to a relation.

Potential Violations:

- ✅ Domain Constraint Violation
- ✅ Key Constraint Violation
- ✅ Entity Integrity Violation
- ✅ Referential Integrity Violation

Example of Successful `INSERT`:

sql Copy Edit

```sql
INSERT INTO STUDENT (Student_ID, Name, Age, Course)
VALUES (101, 'Alice', 20, 'Physics');
```

Update Operations on Relations

SCHOOL OF ENGINEERING

Example of Violations:

❌ **Domain Violation:**

```sql
INSERT INTO STUDENT (Student_ID, Name, Age, Course)
VALUES (102, 'Bob', 'twenty', 'Chemistry');
```

❗ **Error: Age must be an integer (violates domain constraint).**

❌ **Primary Key Violation:**

```sql
INSERT INTO STUDENT (Student_ID, Name, Age, Course)
VALUES (101, 'Eve', 21, 'Maths');
```

❗ **Error: Duplicate `Student_ID` value violates Primary Key constraint.**

Update Operations on Relations

SCHOOL OF ENGINEERING

❌ **Foreign Key Violation:**

sql

Copy Edit

```
INSERT INTO BORROW (Borrow_ID, Student_ID, Book_ID, Borrow_Date)
VALUES (1, 999, 2001, '2025-03-10');
```

❗ **Error:** `Student_ID = 999` **does not exist in the** `STUDENT` **table.**

Update Operations on Relations

SCHOOL OF ENGINEERING

2. DELETE Operation

The `DELETE` command is used to remove records from a table.

Potential Violations:

✅ Referential Integrity Violation

Example of Successful `DELETE`:

sql Copy Edit

```sql
DELETE FROM STUDENT WHERE Student_ID = 102;
```

Update Operations on Relations

SCHOOL OF ENGINEERING

Example of Violation:

❌ **Referential Integrity Violation:**

If the `STUDENT` table has a relationship with `BORROW`, deleting a student who has borrowed books will violate the **Foreign Key** constraint.

sql Copy Edit

```sql
DELETE FROM STUDENT WHERE Student_ID = 101;
```

> ❗ **Error: Cannot delete `Student_ID = 101` because it is referenced in the `BORROW` table.**

Update Operations on Relations

SCHOOL OF ENGINEERING

3. UPDATE Operation

The `UPDATE` command is used to modify attribute values in existing tuples.

Potential Violations:

- ✅ Domain Constraint Violation
- ✅ Key Constraint Violation
- ✅ Entity Integrity Violation
- ✅ Referential Integrity Violation

Example of Successful `UPDATE` :

sql Copy Edit

```sql
UPDATE STUDENT
SET Age = 21
WHERE Student_ID = 101;
```

Update Operations on Relations

SCHOOL OF ENGINEERING

Example of Violations:

❌ **Domain Violation:**

```sql
UPDATE STUDENT
SET Age = 'twenty-one'
WHERE Student_ID = 101;
```

❗ Error: `Age` must be an integer.

❌ **Primary Key Violation:**

```sql
UPDATE STUDENT
SET Student_ID = 102
WHERE Student_ID = 101;
```

❗ Error: Cannot change `Student_ID` to an existing value (violates Primary Key constraint).

Update Operations on Relations

SCHOOL OF ENGINEERING

❌ **Referential Integrity Violation:**

sql

Copy Edit

```
UPDATE BORROW
SET Student_ID = 999
WHERE Borrow_ID = 1;
```

❗ **Error: `Student_ID = 999` does not exist in the `STUDENT` table.**

✅ **Solution for Referential Integrity Violation:**

- Use `ON UPDATE CASCADE` to automatically update related records in child tables.

sql

Copy Edit

```
ALTER TABLE BORROW
ADD CONSTRAINT fk_student
FOREIGN KEY (Student_ID) REFERENCES STUDENT(Student_ID)
ON UPDATE CASCADE;
```

Update Operations on Relations

SCHOOL OF ENGINEERING

4. Strategies to Handle Constraint Violations

Violation Type	Solution / Approach
Domain Violation	Use `CHECK` constraints for valid ranges.
Primary Key Violation	Ensure uniqueness before insertion or update.
Entity Integrity Violation	Avoid NULL values in Primary Key fields.
Referential Integrity Violation	Use `ON DELETE CASCADE` or `ON UPDATE CASCADE`.

SQL Data Definition and Data types

SCHOOL OF ENGINEERING

SQL Data Definition

SQL Data Definition Language (DDL) is used to define, modify, and delete database structures such as tables, indexes, and schemas. DDL commands include:

- `CREATE` — Creates database objects like tables, views, etc.
- `ALTER` — Modifies an existing database structure.
- `DROP` — Deletes database objects like tables or views.
- `TRUNCATE` — Deletes all records from a table without logging individual row deletions.

SQL Data Types

SQL supports various data types for defining the nature of the data to be stored in a database.

Category	Data Type	Description
String	`CHAR(size)`, `VARCHAR(size)`, `TEXT`	Fixed-length and variable-length text data
Numeric	`INT`, `FLOAT`, `DECIMAL(p, s)`, `BIGINT`	Whole numbers, decimals, and large values
Date/Time	`DATE`, `DATETIME`, `TIMESTAMP`, `TIME`	Stores date and time information
Boolean	`BOOLEAN`, `BIT`	TRUE or FALSE values
Binary	`BLOB`, `VARBINARY`	Binary data such as images or files

SQL Data Definition and Data types

SCHOOL OF ENGINEERING

Example - Creating a Table Using DDL and Data Types

sql

Copy Edit

```sql
CREATE TABLE Students (
    StudentID INT PRIMARY KEY,
    Name VARCHAR(50) NOT NULL,
    Age INT CHECK (Age > 18),
    EnrollmentDate DATE,
    GPA DECIMAL(3, 2)
);
```

Students

StudentID	Name	Age	EnrollmentDate	GPA
empty				

SQL Constraints

SCHOOL OF ENGINEERING

2) Specifying Constraints in SQL

SQL constraints are used to enforce rules on the data in tables to ensure data integrity.

Types of Constraints

Constraint	Description
`PRIMARY KEY`	Ensures unique identification of each row
`FOREIGN KEY`	Establishes relationships between tables
`UNIQUE`	Ensures all values in a column are unique
`NOT NULL`	Prevents null values in a column
`CHECK`	Ensures values satisfy specific conditions
`DEFAULT`	Sets a default value if none is provided

SQL Constraints

SCHOOL OF ENGINEERING

1. PRIMARY KEY Constraint

- Ensures each row is uniquely identified.
- Prevents `NULL` and duplicate values.

SQL Query - PRIMARY KEY Example

sql

Copy Edit

```sql
CREATE TABLE Students (
    StudentID INT PRIMARY KEY,
    Name VARCHAR(50) NOT NULL,
    Age INT
);

-- Inserting Data
INSERT INTO Students VALUES (101, 'John Doe', 20);
INSERT INTO Students VALUES (102, 'Alice', 22);

-- Attempting to Insert Duplicate Primary Key (Error Expected)
INSERT INTO Students VALUES (101, 'Bob', 25);

-- Display Data
SELECT * FROM Students;
```

SQL Constraints

SCHOOL OF ENGINEERING

Output:

StudentID	Name	Age
101	John Doe	20
102	Alice	22

❌ **Error:** Duplicate entry for PRIMARY KEY on `INSERT INTO Students VALUES (101, 'Bob', 25);`

SQL Constraints

SCHOOL OF ENGINEERING

2. FOREIGN KEY Constraint

- Establishes relationships between tables.
- Ensures referenced values exist in the parent table.

```sql
CREATE TABLE Departments (
    DeptID INT PRIMARY KEY,
    DeptName VARCHAR(50)
);

CREATE TABLE Employees (
    EmpID INT PRIMARY KEY,
    EmpName VARCHAR(100),
    DeptID INT,
    FOREIGN KEY (DeptID) REFERENCES Departments(DeptID)
);

-- Inserting Data in Parent Table
INSERT INTO Departments VALUES (1, 'HR');
INSERT INTO Departments VALUES (2, 'IT');

-- Inserting Data in Child Table
INSERT INTO Employees VALUES (201, 'John Doe', 1);
INSERT INTO Employees VALUES (202, 'Alice', 2);

-- Attempt to Insert Invalid DeptID (Error Expected)
INSERT INTO Employees VALUES (203, 'Bob', 3);

-- Display Data
SELECT * FROM Employees;
```

Output:

EmpID	EmpName	DeptID
201	John Doe	1
202	Alice	2

❌ **Error:** Cannot add `DeptID = 3` as it does not exist in the `Departments` table.

3. UNIQUE Constraint

- Ensures all values in the column are unique.

SQL Query - UNIQUE Example

sql Copy Edit

```sql
CREATE TABLE Users (
    UserID INT PRIMARY KEY,
    Email VARCHAR(100) UNIQUE
);

-- Inserting Data
INSERT INTO Users VALUES (1, 'john.doe@example.com');
INSERT INTO Users VALUES (2, 'alice@example.com');

-- Attempt to Insert Duplicate Email (Error Expected)
INSERT INTO Users VALUES (3, 'john.doe@example.com');

-- Display Data
SELECT * FROM Users;
```

Output:

UserID	Email
1	john.doe@example.com
2	alice@example.com

❌ **Error:** Duplicate entry for `Email` on `INSERT INTO Users VALUES (3, 'john.doe@example.com');`

4. NOT NULL Constraint

- Ensures specific columns cannot have `NULL` values.

SQL Query - NOT NULL Example

sql

Copy Edit

```sql
CREATE TABLE Products (
    ProductID INT PRIMARY KEY,
    ProductName VARCHAR(50) NOT NULL,
    Price DECIMAL(10, 2) NOT NULL
);

-- Inserting Data
INSERT INTO Products VALUES (1, 'Laptop', 75000);
INSERT INTO Products VALUES (2, NULL, 40000);  -- Error Expected

-- Display Data
SELECT * FROM Products;
```

Output:

ProductID	ProductName	Price
1	Laptop	75000.00

❌ **Error:** `ProductName` cannot be `NULL`.

5. CHECK Constraint

- Ensures column values satisfy specified conditions.

SQL Query - CHECK Example

sql

Copy Edit

```sql
CREATE TABLE Students (
    StudentID INT PRIMARY KEY,
    Name VARCHAR(50),
    Age INT CHECK (Age >= 18),
    GPA DECIMAL(3, 2) CHECK (GPA BETWEEN 0 AND 4.0)
);

-- Inserting Data
INSERT INTO Students VALUES (101, 'John Doe', 20, 3.75);
INSERT INTO Students VALUES (102, 'Alice', 16, 3.0);  -- Error Expected
INSERT INTO Students VALUES (103, 'Bob', 22, 4.5);    -- Error Expected

-- Display Data
SELECT * FROM Students;
```

Output:

StudentID	Name	Age	GPA
101	John Doe	20	3.75

❌ **Error:** Age must be ≥ 18 on `INSERT INTO Students VALUES (102, 'Alice', 16, 3.0);`

❌ **Error:** GPA must be between 0 and 4.0 on `INSERT INTO Students VALUES (103, 'Bob', 22, 4.5);`

6. DEFAULT Constraint

- Assigns a default value when no value is provided.

SQL Query - DEFAULT Example

sql

Copy Edit

```sql
CREATE TABLE Orders (
    OrderID INT PRIMARY KEY,
    OrderDate DATE DEFAULT CURRENT_DATE,
    Status VARCHAR(20) DEFAULT 'Pending'
);

-- Inserting Data
INSERT INTO Orders (OrderID) VALUES (1001);
INSERT INTO Orders VALUES (1002, '2024-02-15', 'Completed');

-- Display Data
SELECT * FROM Orders;
```

Output:

OrderID	OrderDate	Status
1001	2024-03-13	Pending
1002	2024-02-15	Completed

7. Composite Key Example

- Combines two or more columns to ensure uniqueness.

SQL Query - Composite Key Example

sql Copy Edit

```sql
CREATE TABLE Enrollments (
    StudentID INT,
    CourseID INT,
    EnrollmentDate DATE,
    PRIMARY KEY (StudentID, CourseID)
);

-- Inserting Data
INSERT INTO Enrollments VALUES (101, 501, '2024-01-10');
INSERT INTO Enrollments VALUES (101, 502, '2024-01-11');  -- Different CourseID allowed
INSERT INTO Enrollments VALUES (101, 501, '2024-01-15');  -- Error Expected

-- Display Data
SELECT * FROM Enrollments;
```

Output:

StudentID	CourseID	EnrollmentDate
101	501	2024-01-10
101	502	2024-01-11

❌ **Error:** Duplicate entry on `INSERT INTO Enrollments VALUES (101, 501, '2024-01-15');`

Summary Table - Constraint Behavior

Constraint	Allows NULL?	Allows Duplicates?	Key Feature
PRIMARY KEY	❌ No	❌ No	Uniquely identifies rows
FOREIGN KEY	✅ Yes (in child)	✅ Yes (if unique in parent)	Ensures referential integrity
UNIQUE	✅ Yes	❌ No	Ensures distinct values
NOT NULL	❌ No	✅ Yes	Ensures required data
CHECK	✅ Yes	✅ Yes	Ensures data meets conditions
DEFAULT	✅ Yes	✅ Yes	Provides automatic values

SQL Data Definition and Data types

SCHOOL OF ENGINEERING

Example - Adding Constraints to a Table

sql

Copy Edit

```sql
CREATE TABLE Orders (
    OrderID INT PRIMARY KEY,
    CustomerID INT NOT NULL,
    OrderDate DATE DEFAULT CURRENT_DATE,
    Amount DECIMAL(8, 2) CHECK (Amount > 0),
    FOREIGN KEY (CustomerID) REFERENCES Customers(CustomerID)
);
```

SQL Data Definition Language

SCHOOL OF ENGINEERING

Example - DDL Commands

```
CREATE TABLE Employees (
    EmpID INT PRIMARY KEY,
    EmpName VARCHAR(100),
    Salary DECIMAL(8, 2)
);

INSERT INTO Employees VALUES (101, 'Lakshmanan', 20000);


select * from employees;
```

Output

EmpID	EmpName	Salary
101	Lakshmanan	20000

SQL Data Definition Language

SCHOOL OF ENGINEERING

Example - DDL Commands - -- ALTER Table

```
CREATE TABLE Employees (
    EmpID INT PRIMARY KEY,
    EmpName VARCHAR(100),
    Salary DECIMAL(8, 2)
);
INSERT INTO Employees VALUES (101, 'Lakshmanan', 20000);
ALTER TABLE Employees ADD Designation VARCHAR(50);
select * from employees;
```

Output

EmpID	EmpName	Salary	Designation
101	Lakshmanan	20000	

SQL Data Definition Language

SCHOOL OF ENGINEERING

1. DELETE Command

- **Purpose:** Used to remove specific records (rows) from a table.
- **Can include `WHERE` conditions** to delete selected rows.
- **Transaction logs are maintained**, so the action can be rolled back using `ROLLBACK`.

Syntax

sql Copy Edit

```sql
DELETE FROM table_name WHERE condition;
```

SQL Data Definition Language

SCHOOL OF ENGINEERING

Example - DELETE Command

sql

Copy Edit

```sql
-- Sample Table Creation
CREATE TABLE Employees (
    EmpID INT PRIMARY KEY,
    EmpName VARCHAR(100),
    Salary DECIMAL(8, 2),
    Designation VARCHAR(50)
);

-- Inserting Sample Data
INSERT INTO Employees VALUES (101, 'Logesh', 20000, 'Manager');
INSERT INTO Employees VALUES (102, 'Lakshmanan', 25000, 'Developer');
INSERT INTO Employees VALUES (103, 'Kumar', 18000, 'Analyst');

-- Deleting a Specific Record
DELETE FROM Employees WHERE EmpID = 102;

-- Select All Records After DELETE
SELECT * FROM Employees;
```

SQL Data Definition Language

SCHOOL OF ENGINEERING

Output Table After DELETE:

EmpID	EmpName	Salary	Designation
101	Logesh	20000.00	Manager
103	Kumar	18000.00	Analyst

SQL Data Definition Language

SCHOOL OF ENGINEERING

2. TRUNCATE Command

- **Purpose:** Used to delete **all rows** from a table.
- Faster than `DELETE` since it doesn't log individual row deletions.
- **Cannot include a `WHERE` condition.**
- **CANNOT be rolled back** as it commits automatically.

Syntax

sql Copy Edit

```sql
TRUNCATE TABLE table_name;
```

SQL Data Definition Language

SCHOOL OF ENGINEERING

Example - TRUNCATE Command

sql Copy Edit

```sql
-- Truncate the Entire Table
TRUNCATE TABLE Employees;

-- Select All Records After TRUNCATE
SELECT * FROM Employees;
```

Output Table After TRUNCATE:

EmpID	EmpName	Salary	Designation
(Empty Table)			

SQL Data Definition Language

SCHOOL OF ENGINEERING

Key Differences Between DELETE and TRUNCATE

Feature	DELETE	TRUNCATE
Removes	Specific rows using `WHERE`	All rows (no condition allowed)
Rollback	Possible if `BEGIN TRANSACTION` is used	**Not possible** (auto-commit)
Execution Speed	Slower (logs each row deletion)	Faster (minimal logging)
Triggers	Triggers fire when rows are deleted	Triggers do not fire
Resets Identity	**No** — Identity values are retained	**Yes** — Resets identity values

SQL Data Manipulation Language

SCHOOL OF ENGINEERING

(b) Data Manipulation Language (DML) Commands

DML commands are used to manipulate data in the database.

Command	Description
`INSERT`	Adds data to a table
`UPDATE`	Modifies existing data
`DELETE`	Removes specific data from a table
`SELECT`	Retrieves data from one or more tables

```
CREATE TABLE Students (
    StudentID INT PRIMARY KEY,
    Name VARCHAR(50) NOT NULL,
    Age INT CHECK (Age > 18),
    EnrollmentDate DATE,
    GPA DECIMAL(3, 2)
);

INSERT INTO Students (StudentID, Name, Age, EnrollmentDate, GPA)
VALUES (101, 'John Doe', 20, '2024-01-10', 3.75), (102,'Lokesh',30,'2023-02-03',8.0);

-- Step 3: SELECT Data (After INSERT)
SELECT * FROM Students;

-- Step 4: UPDATE Data
UPDATE Students
SET GPA = 3.85
WHERE StudentID = 101;

-- Step 5: SELECT Data (After UPDATE)
SELECT * FROM Students;

-- Step 6: DELETE Data
DELETE FROM Students WHERE StudentID = 101;

-- Step 7: SELECT Data (After DELETE)
SELECT * FROM Students;
```

Output:

StudentID	Name	Age	EnrollmentDate	GPA
101	John Doe	20	2024-01-10	3.75
102	Lokesh	30	2023-02-03	8

StudentID	Name	Age	EnrollmentDate	GPA
101	John Doe	20	2024-01-10	3.85
102	Lokesh	30	2023-02-03	8

StudentID	Name	Age	EnrollmentDate	GPA
102	Lokesh	30	2023-02-03	8

Scenario-Based Questions

SCHOOL OF ENGINEERING

Relational Model Concepts

1. **Scenario:** A university database has a relation `STUDENT` with attributes: `StudentID`, `Name`, `Age`, `Department`, and `GPA`.

   Question: If the tuples in this relation are rearranged by `Name` or `GPA`, would this affect the relation's integrity? Justify your answer.

2. **Scenario:** A company maintains an employee database with the following schema: `EMPLOYEE(EmpID, Name, Age, Salary, DepartmentID)`.

   Question: Identify the primary key, candidate key, and possible foreign key(s) in this relation.

Scenario-Based Questions

SCHOOL OF ENGINEERING

Domain and Attributes

3. **Scenario:** A bank requires a database to manage customer details. The `ACCOUNT` relation has attributes `AccountID`, `CustomerName`, `MobileNumber`, and `Balance`.

   Question: Suggest appropriate domains for each attribute and explain your choice.

Tuples and Relations

4. **Scenario:** Consider the following tuple from a `CUSTOMER` relation:

   `<4562, 'Alice Johnson', 'New York', NULL>`

   Question: What does the NULL value signify, and how should it be handled during data processing?

Scenario-Based Questions

SCHOOL OF ENGINEERING

Relational Model - Keys

5. **Scenario**: A hospital database stores patient details in a relation:

   `PATIENT(PatientID, Name, Age, DoctorID, RoomNo)`.

   Question: Identify the most suitable primary key and suggest another attribute that could act as a candidate key.

6. **Scenario**: An e-commerce platform has two relations:

   - `ORDERS(OrderID, CustomerID, ProductID, OrderDate)`
   - `CUSTOMERS(CustomerID, Name, Address)` **Question**: Which attribute acts as a foreign key, and why is it necessary?

Scenario-Based Questions

SCHOOL OF ENGINEERING

Relational Model Constraints

7. **Scenario:** A retail store wants to ensure that no product price is below zero and no order has a negative quantity.

   Question: Propose suitable constraints for their database to enforce these conditions.

Relational Database Schema

8. **Scenario:** A library management system has a schema with the following relations:

   - `BOOKS(BookID, Title, AuthorID)`
   - `AUTHORS(AuthorID, Name)`

   Question: Design the appropriate primary and foreign keys for this schema.

Scenario-Based Questions

SCHOOL OF ENGINEERING

Update Operations on Relations

9. **Scenario:** In a student management system, the relation `STUDENTS` has the following data:

```markdown
StudentID | Name       | Age | GPA
-----------------------------------
101       | John Doe   | 20  | 3.2
102       | Alice Wang | 22  | 3.5
```

Copy code

Question: Write SQL commands to:

- Insert a new student with ID `103`, named "Robert Smith", aged `21` with a GPA of `3.7`.
- Update `John Doe's` GPA to `3.8`.
- Delete the student entry for `Alice Wang`.

Scenario-Based Questions

SCHOOL OF ENGINEERING

SQL Data Definition Language (DDL)

10. **Scenario:** A travel company requires a database for managing bookings with the following attributes:

- `BookingID (Primary Key)`
- `CustomerID (Foreign Key)`
- `BookingDate`
- `Amount`

Question: Write a SQL `CREATE TABLE` statement for this scenario with appropriate data types and constraints.

Scenario-Based Questions

SCHOOL OF ENGINEERING

SQL Data Manipulation Language (DML)

11. **Scenario:** A school's database contains a `STUDENTS` relation with the following structure:

    `STUDENTS(StudentID, Name, Age, GPA)`

    Question: Write SQL queries to:

- Retrieve all students with GPA greater than 3.5.
- Increase the GPA of all students aged 20 by 0.5.
- Display students sorted by their GPA in descending order.

Scenario-Based Questions

SCHOOL OF ENGINEERING

SQL Constraints

12. **Scenario**: A warehouse database tracks product details. The `PRODUCTS` relation includes:

    `ProductID, ProductName, Quantity, Price`.

    Question: Define suitable constraints to:

- Ensure `Quantity` is always greater than zero.
- Restrict `Price` to values above `$1`.

Scenario-Based Answers

SCHOOL OF ENGINEERING

1. Relational Model Concepts

Scenario: Rearranging tuples in a `STUDENT` relation by `Name` or `GPA`

Answer:

Explanation: The relational model is **not sensitive to tuple order**; thus, rearranging rows has no effect on the relation's integrity.

Scenario-Based Answers

SCHOOL OF ENGINEERING

2. Primary Key, Candidate Key, and Foreign Key

Scenario: `EMPLOYEE(EmpID, Name, Age, Salary, DepartmentID)`

sql

Copy code

```sql
-- Primary Key
ALTER TABLE EMPLOYEE ADD PRIMARY KEY (EmpID);

-- Candidate Key (If 'Name' is unique, it can also be a candidate key)
ALTER TABLE EMPLOYEE ADD UNIQUE (Name);

-- Foreign Key (Assuming DEPARTMENT table exists)
ALTER TABLE EMPLOYEE
ADD CONSTRAINT fk_department
FOREIGN KEY (DepartmentID)
REFERENCES DEPARTMENT(DepartmentID);
```

Scenario-Based Answers

SCHOOL OF ENGINEERING

3. Domains for Attributes

Scenario: Defining appropriate domains

sql

Copy code

```sql
CREATE TABLE ACCOUNT (
    AccountID INT PRIMARY KEY,
    CustomerName VARCHAR(100) NOT NULL,
    MobileNumber CHAR(10) CHECK (MobileNumber LIKE '[0-9]%'),
    Balance DECIMAL(10, 2) CHECK (Balance >= 0)
);
```

Scenario-Based Answers

SCHOOL OF ENGINEERING

4. Handling NULL Values

Scenario: `CUSTOMER` relation with NULL values

Answer: Use `IS NULL` or `IS NOT NULL` in queries.

sql

Copy code

```sql
-- Select customers with unknown addresses
SELECT * FROM CUSTOMER WHERE Address IS NULL;

-- Update NULL values with default data
UPDATE CUSTOMER SET Address = 'Unknown' WHERE Address IS NULL;
```

Scenario-Based Answers

SCHOOL OF ENGINEERING

5. Primary Key and Candidate Key

Scenario: Identifying keys for `PATIENT` relation

sql

Copy code

```sql
-- Primary Key
ALTER TABLE PATIENT ADD PRIMARY KEY (PatientID);

-- Candidate Key
ALTER TABLE PATIENT ADD UNIQUE (DoctorID);
```

Scenario-Based Answers

SCHOOL OF ENGINEERING

6. Foreign Key Identification

Scenario: Identifying foreign keys in `ORDERS` and `CUSTOMERS`

sql

Copy code

```sql
ALTER TABLE ORDERS
ADD CONSTRAINT fk_customer
FOREIGN KEY (CustomerID)
REFERENCES CUSTOMERS(CustomerID);
```

Scenario-Based Answers

SCHOOL OF ENGINEERING

7. Relational Model Constraints

Scenario: Enforcing conditions on product prices and order quantity

sql

Copy code

```sql
CREATE TABLE PRODUCTS (
    ProductID INT PRIMARY KEY,
    ProductName VARCHAR(100) NOT NULL,
    Quantity INT CHECK (Quantity >= 0),
    Price DECIMAL(10, 2) CHECK (Price > 0)
);
```

Scenario-Based Answers

SCHOOL OF ENGINEERING

8. Relational Database Schema

Scenario: Library management schema

sql

Copy code

```sql
CREATE TABLE AUTHORS (
    AuthorID INT PRIMARY KEY,
    Name VARCHAR(100) NOT NULL
);

CREATE TABLE BOOKS (
    BookID INT PRIMARY KEY,
    Title VARCHAR(150) NOT NULL,
    AuthorID INT,
    FOREIGN KEY (AuthorID) REFERENCES AUTHORS(AuthorID)
);
```

Scenario-Based Answers

SCHOOL OF ENGINEERING

9. Update Operations

Scenario: Performing `INSERT`, `UPDATE`, and `DELETE` operations

sql

Copy code

```sql
-- Insert new student
INSERT INTO STUDENTS (StudentID, Name, Age, GPA)
VALUES (103, 'Robert Smith', 21, 3.7);

-- Update GPA for John Doe
UPDATE STUDENTS SET GPA = 3.8 WHERE StudentID = 101;

-- Delete Alice Wang's record
DELETE FROM STUDENTS WHERE StudentID = 102;
```

Scenario-Based Answers

SCHOOL OF ENGINEERING

10. SQL Data Definition Language (DDL)

Scenario: Creating a `BOOKINGS` table

sql

Copy code

```sql
CREATE TABLE BOOKINGS (
    BookingID INT PRIMARY KEY,
    CustomerID INT NOT NULL,
    BookingDate DATE NOT NULL,
    Amount DECIMAL(10, 2) CHECK (Amount > 0),
    FOREIGN KEY (CustomerID) REFERENCES CUSTOMERS(CustomerID)
);
```

Scenario-Based Answers

SCHOOL OF ENGINEERING

11. SQL Data Manipulation Language (DML)

Scenario: Performing complex queries on `STUDENTS`

sql

Copy code

```sql
-- Retrieve students with GPA greater than 3.5
SELECT * FROM STUDENTS WHERE GPA > 3.5;

-- Increase GPA of students aged 20 by 0.5
UPDATE STUDENTS SET GPA = GPA + 0.5 WHERE Age = 20;

-- Display students sorted by GPA in descending order
SELECT * FROM STUDENTS ORDER BY GPA DESC;
```

Scenario-Based Answers

SCHOOL OF ENGINEERING

12. SQL Constraints for PRODUCT Table

Scenario: Enforcing conditions on `PRODUCTS` relation

sql

Copy code

```sql
CREATE TABLE PRODUCTS (
    ProductID INT PRIMARY KEY,
    ProductName VARCHAR(100) NOT NULL,
    Quantity INT CHECK (Quantity > 0),
    Price DECIMAL(10, 2) CHECK (Price > 1)
);
```


Thank you!